Deliverable II: AI Fake Video Detection Through Cloth Simulation

Final Year Project

Contents

1 Introduction **1**

2 Project Vision **2**

3 Literature Review and Related Applications **3**

3.1 Physical limitations of generative video models....3
3.2 Appearance-based deepfake and synthetic-video detection....3
3.3 Physical and geometric consistency as a generalisable signal....3
3.4 Cloth simulation and physical modelling....3
3.5 Segmentation and tracking tools....4
3.6 Related applications....4
3.7 Positioning of this project....4

4 Software Requirement Specifications **5**

4.1 List of Features....5
4.2 Functional Requirements....5
4.3 Quality Attributes....6
4.3.1 Reliability....6
4.3.2 Maintainability....6
4.3.3 Usability....7
4.3.4 Correctness....7
4.3.5 Explainability....7
4.4 Non-Functional Requirements....7
4.4.1 Performance....7
4.4.2 Portability....7
4.4.3 Reproducibility....7
4.4.4 Robustness....8
4.4.5 Security and Privacy....8
4.5 Assumptions....8
4.6 Use Cases....8
4.6.1 Prepare Dataset and Extract Frames....9
4.6.2 Annotate Garment Boxes....10
4.6.3 Segment Garments....11

4.6.4 Extract Body Pose 11
4.7 Hardware and Software Requirements 12
4.7.1 Hardware Requirements 12
4.7.2 Software Requirements 12
4.8 Graphical User Interface 12
4.9 Data Design 18
4.10 Risk Analysis 18
4.10.1 Technical Risks 19
4.10.2 Research Risks 19
4.10.3 Ethical and Legal Risks 19
4.10.4 Schedule Risks 19
4.11 Conclusion 20

5 Proposed Approach and Methodology 21
5.1 Pipeline Overview 21
5.2 Research Hypothesis 22
5.3 Dataset 22
5.4 Methodology per Stage 23
5.4.1 Frame Extraction 23
5.4.2 Garment Segmentation 23
5.4.3 Body Pose Estimation 24
5.4.4 Classification and Evaluation 24
5.4.5 Explainability 25
5.5 Development Methodology 25
5.6 Conclusion 25

6 High-Level and Low-Level Design 26
6.1 System Overview 26
6.1.1 System Design 26
6.1.2 Data Flow 26
6.2 Design Considerations 27
6.2.1 Assumptions and Dependencies 27
6.2.2 General Constraints 27
6.2.3 Goals and Guidelines 27
6.2.4 Development Methods 28

6.3 System Architecture 28
6.3.1 Subsystem Architecture 29
6.4 Architectural Strategies 29
6.4.1 Pipe-and-Filter with File-Based Contracts 29
6.4.2 Technology Stack 29
6.4.3 Reuse of Pretrained Foundation Models 30
6.4.4 Offloading to the Cloud 30
6.4.5 Error Handling and Recovery 30
6.4.6 Future Extension 30
6.5 Component Diagram 30
6.6 Sequence Diagrams 31
6.7 Policies and Tactics 35
6.7.1 Frame Rate of 30 Frames per Second 35
6.7.2 Box Prompts for Segmentation 36
6.7.3 Visual Review of Masks and Pose 36
6.7.4 Coding Guidelines 36
6.8 Conclusion 36

List of Figures

4.1 Use Case Diagram 9
4.2 Upload Video Screen 13
4.3 Mark the Garments Screen 14
4.4 Analysis Screen: Isolating the Clothes and the Body 15
4.5 Analysis Screen: Simulating the Cloth 16
4.6 Result Screen: Real Video 17
4.7 Result Screen: AI-Generated Video 18

5.1 Overview of the Proposed Methodology 22
5.2 Garment Segmentation and Point Tracking 24

6.1 High-Level System Architecture 28
6.2 Perception Subsystem Architecture 29
6.3 Component Diagram of Phases 1 and 2 31
6.4 Sequence Diagram: End-to-End Pipeline (Planned Stages) 32
6.5 Sequence Diagram: Frame Extraction 33
6.6 Sequence Diagram: Garment Segmentation 34
6.7 Sequence Diagram: Pose Estimation 35

Chapter 1 Introduction

Generative video models such as Sora, Runway, Kling, Pika, and Veo have advanced at an extraordinary pace. In a short span of time, AI-generated video has moved from obviously artificial clips to footage that, at normal playback speed, is almost indistinguishable from genuine recordings. While this technology enables many positive applications, it also makes it dangerously easy to fabricate convincing videos of real-looking people and events. As generation quality keeps improving, the central question this project addresses becomes more urgent: given a video, how can we reliably tell whether it was captured by a real camera or synthesised by an AI model?

The dominant detection strategy today is appearance-based: a neural network is trained to recognise the visual artifacts a specific generator tends to leave behind, such as blurred edges, flickering textures, or warped backgrounds. The weakness of this strategy is that it is tightly coupled to the generator it was trained on. When a new model is released, or an existing one is improved, the old artifacts disappear and the detector's accuracy collapses. Detection is therefore always one step behind generation, and this gap keeps widening.

This project takes a different position. Instead of chasing cosmetic artifacts that change from model to model, it exploits a property no generator can opt out of: real footage always obeys the laws of physics, while current generators have no physical engine and merely predict the next set of pixels. As the physical "probe" for this, the project uses cloth. Fabric is non-rigid, has no fixed geometry, and reacts continuously to gravity, body motion, and collision with the body underneath it. These properties make cloth highly informative, yet very difficult for a generator to fake convincingly — a documented failure of current video models is that clothing sometimes moves independently of the body, stretches unnaturally, or produces folds that appear and disappear between frames.

The overall approach combines calculation and prediction: from a video, the system extracts the region of clothing on a moving person, measures how the fabric deforms across frames using physics-based formulas and a driven cloth simulation, and feeds these measurements into a machine-learning classifier that decides whether the video is real or AI-generated. The system is also designed to report a confidence score and the timestamps at which the cloth behaves implausibly, so its decision can be inspected and explained rather than taken on trust.

This document is Deliverable II of the project and is prepared as **Phase 2** nears completion. The project is delivered in five phases: Phase 1, frame extraction; Phase 2, garment segmentation with SAM2 and body-pose extraction with MediaPipe; Phase 3, point tracking on the segmented garments; Phase 4, body-driven cloth simulation and physics feature extraction; and Phase 5, classification and reporting. Chapters 2 to 6 describe the full planned system and the work completed up to the end of Phase 2.

Chapter 2 Project Vision

The long-term vision of this project is a **generator-agnostic, explainable detector of fully AI-generated human videos**, built on a signal that does not erode as generative models improve: physical plausibility rather than visual polish.

Most detectors in production today are trained to recognise the visual signature of a specific generation pipeline. They perform well until the next model version ships, at which point their accuracy quietly collapses — a pattern already visible across the deepfake-detection literature. This project is guided by the belief that a durable detector cannot be built purely on appearance; it must be anchored to something a generator cannot simply learn away, because that thing was never part of what the generator was trained to reproduce in the first place. Cloth physics is that anchor: a text-to-video model is trained to predict plausible-looking pixels, not to solve Newton's equations of motion for a woven fabric draped over a moving body. Even as image fidelity improves, there is no guarantee that physical consistency improves with it — evidence so far suggests the opposite, that photorealism and physical understanding are only weakly linked in current systems.

Beyond raw accuracy, the vision includes **explainability**: rather than returning a bare real/fake label, the system is intended to point at *when* and *where* the clothing in a video violates physical expectations (a specific time range, a specific kind of violation), so a journalist, platform moderator, or forensic reviewer can inspect the evidence behind the verdict instead of trusting an opaque score.

The intended real-world contexts for this work include:

- Supporting **content moderation** on social platforms where fabricated human videos can spread misinformation.
- Assisting **journalists and fact-checkers** verifying footage before publication.
- Providing an **auxiliary forensic signal** for investigators evaluating video evidence, used alongside — not instead of — existing methods.
- Serving as a **research direction** that other detection work can build on, since a physics-based signal is expected to transfer across generators better than an appearance-based one.

Ultimately, success for this project is not just "a classifier with high accuracy on one dataset," but a detector whose accuracy holds up on generators it has never seen, because it is checking something those generators were never built to get right.

Chapter 3 Literature Review and Related Applications

3.1 Physical limitations of generative video models

The Physics-IQ benchmark [1] evaluates leading video generators — including Sora, Runway, Pika, Lumiere, and Stable Video Diffusion — on their grasp of physical principles, and finds that physical understanding is severely limited and largely decoupled from visual realism: even the best-performing model achieved only around a quarter of the achievable physics score. Independent inspections of Sora-generated footage report the same pattern anecdotally — fabric and other deformable materials frequently move in physically implausible ways, including clothing that moves independently of the body beneath it. This is the central gap the project is built to exploit.

3.2 Appearance-based deepfake and synthetic-video detection

Most published detectors are appearance-based. Sugiyama and Kataoka [2] train convolutional networks to recognise categories of visual artifacts specific to Sora-generated frames. FaceForensics++ [3] established the standard large-scale benchmark and training pipeline for learning manipulation artifacts in classical face-swap deepfakes. These approaches can perform very well on the generator (or manipulation method) they were trained against, but the accuracy is not portable: a newer or simply different generator produces different artifacts, and detection performance degrades accordingly. This generalisation gap is the applied problem this project is responding to.

3.3 Physical and geometric consistency as a generalisable signal

More directly relevant to this project's premise, Sarkar et al. [4] show that physical and geometric inconsistencies — impossible shadows, projective-geometry errors — are a reliable and comparatively generator-agnostic signal that a video is synthetic. This supports the core hypothesis of this project: that physics-based cues, unlike cosmetic artifacts, are a property of *what the generator fundamentally does not model*, and should therefore generalise better across different generators than appearance-based fingerprints.

3.4 Cloth simulation and physical modelling

Classical computer-graphics literature supplies the mathematical models used to describe how real fabric is expected to move. Established cloth-simulation models [5, 6] describe gravity, stretch resistance, bending, and collision response for cloth, and provide the reference physics this project's simulation

stage is built on: given a body's real motion, these models predict how genuine cloth attached to that body should move.

3.5 Segmentation and tracking tools

Modern video segmentation, pose-estimation and point-tracking tools make isolating and following a garment across frames practical. Segment Anything 2, SAM2 [7], provides promptable, temporally consistent segmentation across video frames; this project uses it in Phase 2 to segment each subject's upper and lower garments. Body pose is estimated with Google's MediaPipe Pose, which returns 33 body landmarks per frame and describes how the body under the clothing moves. A point-tracking model is used in the following phase to follow individual points on the segmented garments across frames, which is what makes frame-to-frame physical measurement possible.

3.6 Related applications

Beyond direct deepfake detection, the same class of physics-consistency reasoning has applications in: automated video forensics for legal and journalistic verification, provenance/authenticity tooling for social platforms, quality-control tooling for the generative-video industry itself (flagging physically implausible outputs before release), and academic benchmarking of how "physically aware" a given generator is (in the spirit of Physics-IQ, but specialised to a single, information-rich material: cloth).

3.7 Positioning of this project

As far as the project's review of the literature shows, no existing published work uses cloth physics specifically as a generator-agnostic discriminator for AI-generated human videos — most physics-based detection work looks at scene-level cues such as shadows and geometry, not at deformable-material dynamics. This is the specific gap this project addresses.

Chapter 4 Software Requirement Specifications

This chapter specifies the requirements of the system that detects AI-generated videos of people by testing whether the motion of their clothing obeys physical law. It lists the features of the system, its functional requirements, the quality attributes and non-functional requirements it must satisfy, the assumptions it rests on, its use cases, the hardware and software it needs, the proposed graphical user interface, the design of the data it produces, and the risks the project faces.

The system is built as a sequence of processing stages (a *pipeline*). Each stage is a separate Python program that reads the output of the previous stage from disk and writes its own output for the next. The project is delivered in five phases. Phase 1 (frame extraction) is complete and Phase 2 (garment segmentation and pose extraction) is at the point of completion; this chapter specifies these two phases. Phases 3 to 5 (point tracking, cloth simulation and feature extraction, and classification and reporting) are planned and fall outside these requirements.

4.1 List of Features

The following features are available in the system. The phase that delivers each feature is shown in brackets.

- Extraction of video frames at a fixed rate of 30 frames per second, resized to a standard width (Phase 1).
- Interactive annotation of the upper and lower garment of the subject with bounding boxes (Phase 2).
- Segmentation of each garment in every frame with the SAM2 video segmentation model (Phase 2).
- Estimation of the subject's body pose (33 skeleton landmarks) in every frame with MediaPipe Pose (Phase 2).
- Visual overlays of masks and skeletons on the frames for manual review (Phase 2).
- A Google Colab notebook that runs the GPU-heavy stages on a free cloud GPU.

4.2 Functional Requirements

The following functional requirements shall be available in the system.

FR-1 The system shall read real and fake videos from a dataset folder in the formats .mp4, .mov, .avi, .mkv and .webm.

FR-2 The system shall extract frames from each video at a configurable rate (30 frames per second by default for this project) and resize them to a configurable width (512 pixels).

FR-3 The system shall store the frames of each video as a numbered sequence that the segmentation model can read in order.

FR-4 The system shall let the user draw one bounding box around the upper garment and one around the lower garment on the first frame of each video, and shall save these boxes.

FR-5 The system shall segment the upper and lower garment in every frame from the saved boxes and save one binary mask per garment per frame.

FR-6 The system shall estimate 33 body landmarks per frame and save their pixel coordinates.

FR-7 The system shall save annotated images that overlay the garment masks and the body skeleton on the video frames, so that every video's results can be reviewed.

FR-8 The system shall log any video that fails at a stage and continue with the remaining videos.

4.3 Quality Attributes

The project aims to provide the following quality attributes. Each one is used to judge the quality of the system during development and operation.

4.3.1 Reliability

The system processes datasets in batches, and one problem video must not stop a batch that may take hours on a GPU. Every stage catches the failure of a single video, writes it to a failure log, and continues with the next video. Each stage writes its output to disk, so a stage can be re-run on its own without repeating the stages before it.

4.3.2 Maintainability

Each processing stage is a separate program with a single responsibility, and the stages communicate only through stored data with a fixed structure (images and structured records). A stage can therefore be replaced without changing the others; for example, the segmentation model can be exchanged for another that produces garment masks in the same format.

4.3.3 Usability

Every program can be run with its default arguments from the project folder, and prints its progress per video. The only manual step is drawing two boxes per video, on the first frame.

4.3.4 Correctness

Every later stage builds on the frames, garment masks and body landmarks produced in Phases 1 and 2, so these must be accurate and must be produced in the same way for every video. The system therefore:

- extracts every video at the same frame rate and width, so that all videos are processed alike;
- stores frames in a fixed numbered order, so that no frame is skipped or read out of sequence;
- segments each garment from an explicit bounding box rather than a single point, so that the mask covers the garment and nothing else;
- applies exactly the same processing to real and AI-generated videos.

4.3.5 Explainability

Every result of Phases 1 and 2 can be checked by eye: the system draws the garment masks and the body skeleton over the original frames, so a reviewer can see exactly which pixels were taken as clothing and where the body joints were placed.

4.4 Non-Functional Requirements

4.4.1 Performance

- The system shall run every stage on a laptop-class GPU with around 8 GB of video memory.

4.4.2 Portability

- The system shall run on Windows with PowerShell, with Python 3.10 or later.
- The GPU stages shall also run on Google Colab through the provided notebook.

4.4.3 Reproducibility

- All library versions shall be fixed in a dependency list so that the environment can be rebuilt.
- Every parameter that affects the result shall be configurable, with a documented default.
- Every issue met and its resolution shall be recorded in the project’s troubleshooting log.

4.4.4 Robustness

- The system shall handle frames in which no pose is detected or a garment mask is empty, by recording a missing value rather than stopping.

4.4.5 Security and Privacy

- All processing shall take place on the user's machine or the user's own Colab session. No video shall be sent to any third-party service.
- The dataset and all derived data shall be excluded from the public code repository.

4.5 Assumptions

The assumptions made for the specification of the system are:

- Each video shows one person whose upper and lower garments are visible for most of the video.
- The camera is roughly still, or moves slowly compared to the subject.
- The user has access to an NVIDIA GPU with CUDA support, or to Google Colab.
- The user can draw two bounding boxes per video on the first frame.
- The user supplies the dataset of real and AI-generated videos, and has the right to use it.

4.6 Use Cases

Figure 4.1 shows the actors of the system and the use cases they take part in. The *Researcher* runs the pipeline stages, and the pretrained models (SAM2 and MediaPipe) are external actors that the system calls. The four use cases cover Phases 1 and 2.

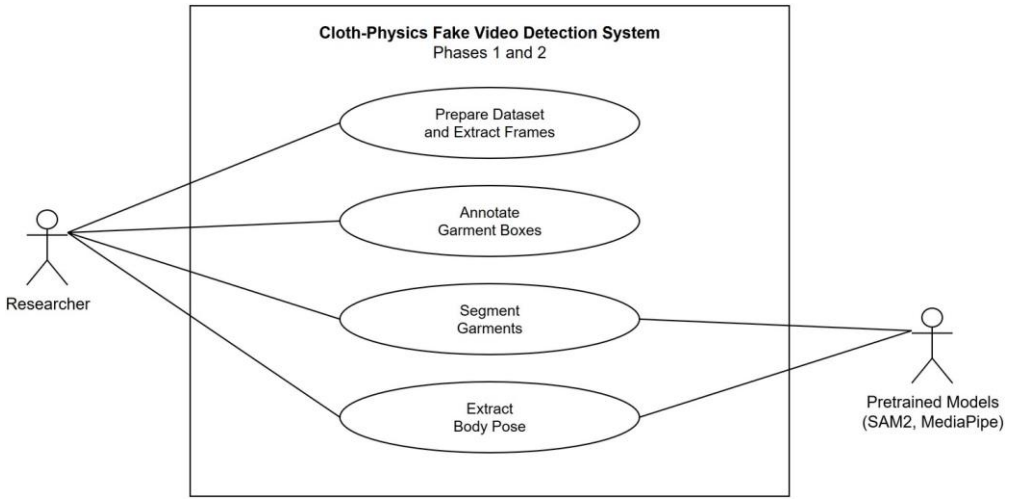


Figure 4.1: Use Case Diagram

Actors of the system and the use cases of Phases 1 and 2 they take part in.

4.6.1 Prepare Dataset and Extract Frames

Name	Prepare Dataset and Extract Frames		
Actors	Researcher		
Summary	The researcher places real and fake videos in the dataset folder and extracts their frames.		
Pre-Conditions	Real and AI-generated videos are placed in separate folders of the dataset.		
Post-Conditions	One folder of numbered frames exists for every video.		
Special Requirements	Video file names must not end with a space.		
Basic Flow			
Actor Action		**System Response**	
1	Researcher runs the frame extraction program with the target rate and width.	2	System lists the videos in both class folders.
		3	System reads each video, samples frames at 30 fps, resizes them and saves them as numbered JPEG files.
		4	System prints the number of frames saved per video.
Alternative Flow			
1	The video cannot be opened.	3-A	System reports the error, skips the video and continues.
1	The video name ends with a space.	3-B	System strips the space from the output folder name so the frames are written correctly.

4.6.2 Annotate Garment Boxes

Name	Annotate Garment Boxes		
Actors	Researcher		
Summary	The researcher marks the upper and lower garment of each video with a bounding box.		
Pre-Conditions	Frames have been extracted. A display is available.		
Post-Conditions	Two garment boxes are stored for every annotated video.		
Special Requirements	Must run on a local machine with a screen.		
Basic Flow			
Actor Action		**System Response**	
1	Researcher runs the annotation program.	2	System shows the first frame of the next unannotated video.
3	Researcher drags a box around the upper garment, then the lower garment.	4	System draws the boxes and saves them under the video's name.
Alternative Flow			
3	Researcher wants to correct an earlier video.	4-A	With the redo option, System shows annotated videos again and overwrites their boxes.
3	Researcher skips a video.	4-B	System leaves the video unannotated; later stages skip it.

4.6.3 Segment Garments

Name	Segment Garments		
Actors	Researcher, SAM2 model		
Summary	The system produces a mask of each garment in every frame.		
Pre-Conditions	Frames have been extracted and the garment boxes drawn. The SAM2 checkpoint is downloaded.		
Post-Conditions	A mask of the upper and of the lower garment is stored for every frame.		
Special Requirements	GPU with CUDA support.		
Basic Flow			
Actor Action		**System Response**	
1	Researcher runs the segmentation program with the checkpoint and configuration.	2	System loads the SAM2 video predictor.
		3	For each video, System gives the two boxes to SAM2 on the first frame and propagates the masks through the video.
		4	System saves each mask and a sanity-check overlay image.
Alternative Flow			
1	The configuration is given as an absolute path.	2-A	System reports a configuration error; the researcher passes the package-relative name instead.
		3-A	GPU runs out of memory: System logs the video as failed and continues.

4.6.4 Extract Body Pose

Name	Extract Body Pose		
Actors	Researcher, MediaPipe Pose		
Summary	The system estimates the skeleton of the subject in every frame.		
Pre-Conditions	Frames have been extracted.		
Post-Conditions	One pose file per video holds 33 landmarks per frame.		
Special Requirements	Compatible versions of the pose library and its dependencies.		
Basic Flow			
Actor Action		**System Response**	
1	Researcher runs the pose extraction program.	2	System runs MediaPipe Pose on every frame.
		3	System converts the landmarks to pixel coordinates and saves them.
Alternative Flow			
		2-A	No person is detected in a frame: System stores an empty entry for that frame and continues.

4.7 Hardware and Software Requirements

The project employs the following hardware and software.

4.7.1 Hardware Requirements

- A computer with an NVIDIA GPU supporting CUDA. The Small SAM2.1 model fits laptop-class GPUs with around 8 GB of video memory. Google Colab's free GPU can be used instead.
- At least 16 GB of system memory.
- About 10 GB of free disk space for libraries and model checkpoints, plus space for the dataset and the generated data (frames, masks and pose landmarks).
- A display, for the garment annotation step.

4.7.2 Software Requirements

- Windows with PowerShell (or Google Colab)
- Python, with a virtual environment
- PyTorch with GPU (CUDA) support
- SAM 2.1 for garment segmentation
- MediaPipe for pose estimation
- OpenCV, NumPy, SciPy and Matplotlib
- Git and GitHub, Visual Studio Code, and draw.io for the design diagrams

4.8 Graphical User Interface

The pipeline is currently operated by the researcher. For Phase 5, a reporting interface named *DripDetector* has been designed so that a non-technical user can test a video. The screens below are taken from its interactive prototype, designed in Figma. They show the planned user journey: uploading a video, marking the garments, following the analysis through its five stages, and reading the verdict. They are designs of the planned interface, not screenshots of a finished application.

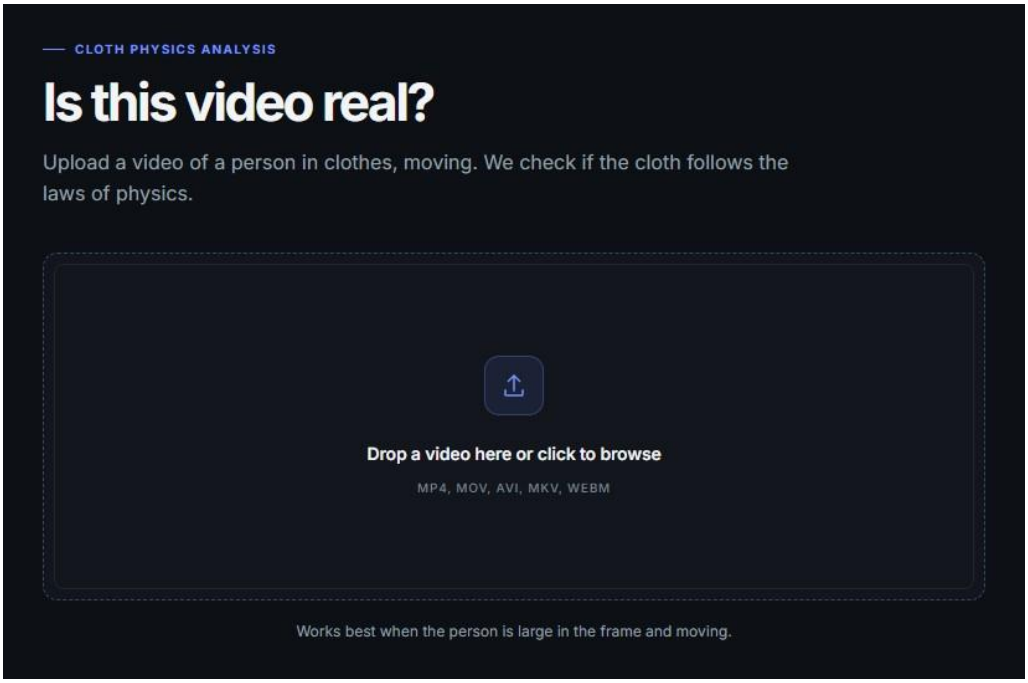


Figure 4.2: Upload Video Screen

The user drops or browses for a video of a clothed person in motion; the supported formats are listed, with a hint that the method works best when the person is large in the frame and moving.

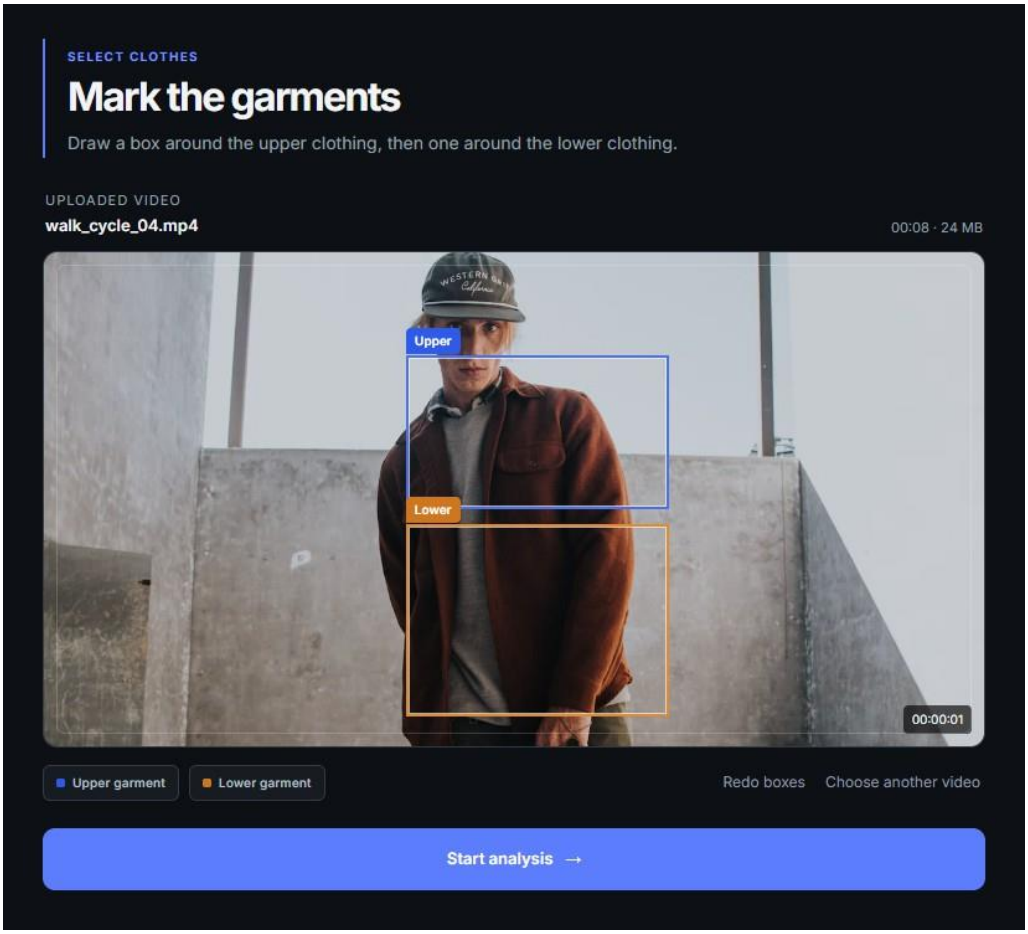


Figure 4.3: Mark the Garments Screen

The first frame of the uploaded video is shown and the user draws one box around the upper and one around the lower garment, then starts the analysis. Covers the Annotate Garment Boxes use case.



Figure 4.4: Analysis Screen: Isolating the Clothes and the Body

Progress through the five stages. Capture is done; Isolate has cut out the upper and lower garment and found the 33 body joints, and is running its quality check. Covers the Prepare Dataset and Extract Frames, Segment Garments and Extract Body Pose use cases.



Figure 4.5: Analysis Screen: Simulating the Cloth

Capture, Isolate and Follow are done; Simulate has built the cloth mesh, attached it to the body and run the physics simulation, and is measuring the difference in stretch, drape and acceleration.

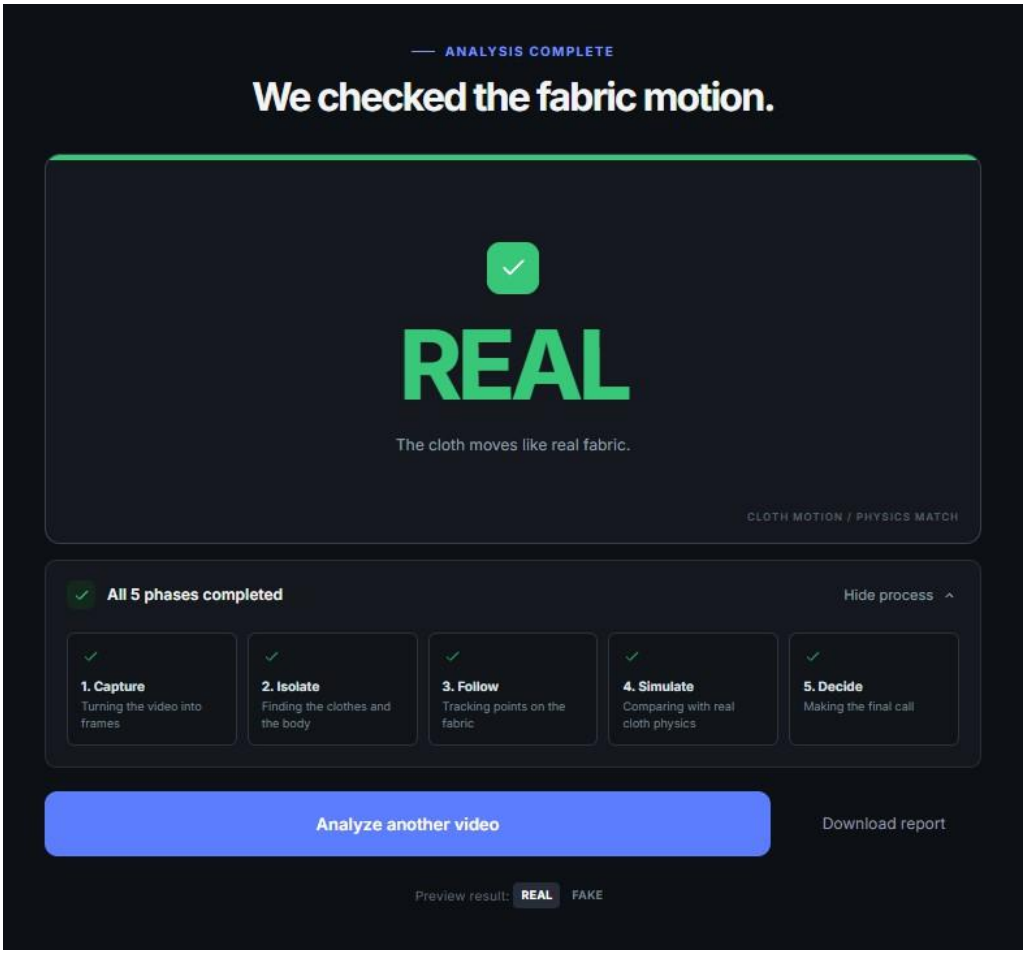


Figure 4.6: Result Screen: Real Video

The verdict for a video whose cloth moves like real fabric, with the five completed stages expanded and options to analyse another video or download a report.

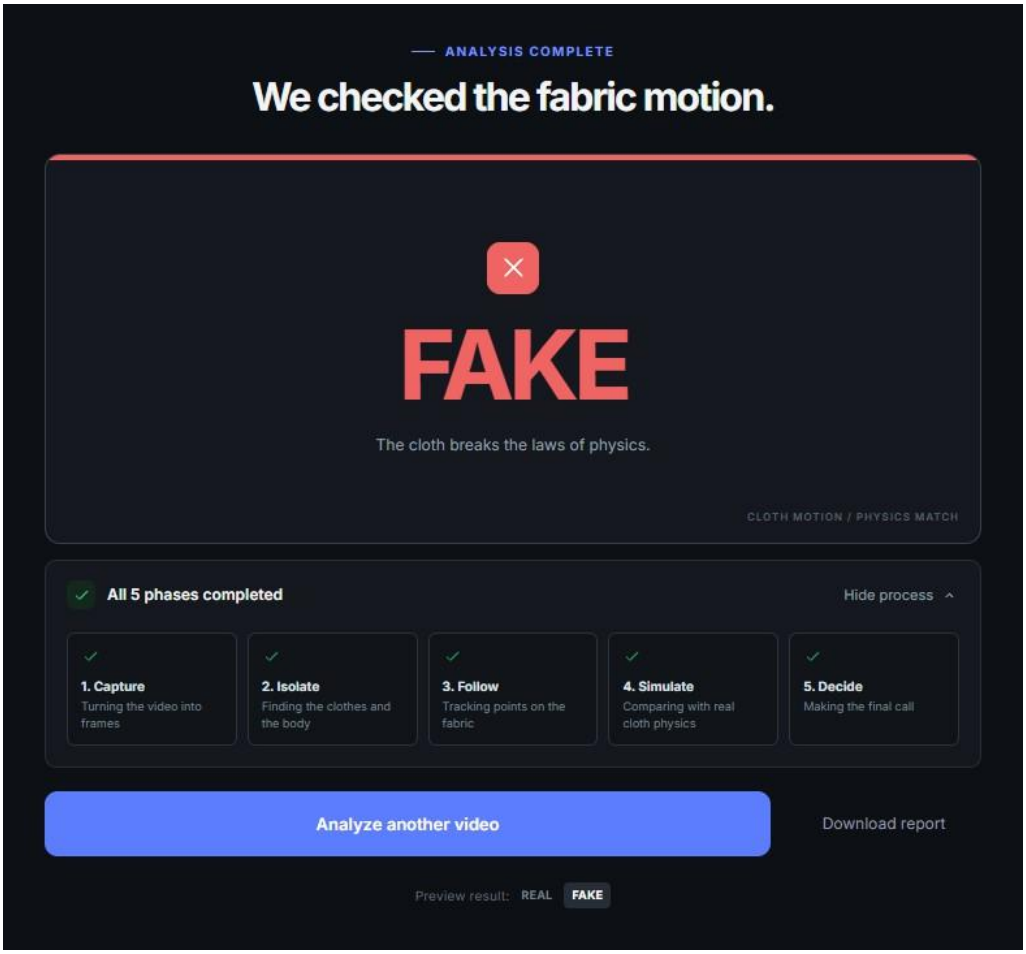


Figure 4.7: Result Screen: AI-Generated Video

The verdict for a video whose cloth breaks the laws of physics.

4.9 Data Design

The system does not use a database server. Each stage stores its results in an organised folder structure, grouped by class (real or AI-generated) and by video, and the next stage reads them from there. This keeps every intermediate result open to inspection and lets any stage be re-run on its own without repeating the stages before it.

4.10 Risk Analysis

Detecting fake videos through physics is a research question, not only an engineering task, so the project carries research risks as well as technical ones. The main risks and how they are addressed are described below.

4.10.1 Technical Risks

- **Small subjects and 2D video.** When the subject is small in the frame, small errors in the masks and landmarks are a large share of the garment, and the video shows 3D cloth only as 2D image motion. Mitigation: a minimum subject size in the dataset, and a visual review of every video's masks and pose.
- **GPU memory.** The segmentation model runs close to the limit of a laptop-class GPU. Mitigation: the Small SAM2.1 model, processing one video at a time and releasing memory after each, and the Colab alternative.
- **Environment and library problems.** The pose library needs specific versions of its supporting libraries, and in Phase 2 a stuck GPU driver state after a crash produced false out-of-memory errors. Mitigation: pinned versions, a documented installation order, and the troubleshooting log.

4.10.2 Research Risks

The central risk is that the physics features may not separate real from fake videos. The current dataset is small (9 real and 5 AI-generated videos); if the subjects are small in frame, wear mostly tight clothing with little free-hanging fabric, or are framed differently in the two classes, a classifier may learn these differences instead of physics. The mitigation is the dataset methodology of Chapter 5: matched real and AI-generated videos across clothing types and motion types, from several generators, with subjects large enough in frame for reliable tracking.

4.10.3 Ethical and Legal Risks

- **False accusations.** Labelling a genuine video as fake can harm the person in it. The system therefore reports evidence with every verdict and is not intended as the sole basis of a decision.
- **Data rights and consent.** Videos of real people may be subject to copyright and privacy law. Only videos the project has the right to use are kept, and none are published in the repository.
- **Misuse.** Published detection features could help generator developers avoid detection. The features are described at the level needed for evaluation.

4.10.4 Schedule Risks

Environment problems consumed significant time in Phases 1 and 2. The risk to the remaining phases is reduced by the Colab notebook, the documented setup guide and the troubleshooting log.

4.11 Conclusion

This chapter specified the requirements of the system: its features, its functional requirements, the quality attributes and non-functional requirements it must meet, its assumptions, the four use cases of Phases 1 and 2, its hardware and software needs, the proposed interface and the design of its data. The risk analysis identifies the dataset as the main research risk and sets out how it is addressed. These requirements form the basis of the approach in Chapter 5 and the design in Chapter 6.

Chapter 5 Proposed Approach and Methodology

The system is organised as an end-to-end pipeline in which each stage produces the input the next stage needs. The methodology is being delivered in phases; this chapter describes the full intended pipeline and states which parts are complete as of this deliverable.

This chapter describes how the system decides whether the clothing in a video moves as real cloth would. The approach turns a video into frames, finds the garments and the body, follows points on the garments through time, predicts with a physics simulation how those points should have moved given the real body motion, and measures how far the observed motion departs from physical law. The sections that follow explain the stages of Phases 1 and 2 in detail, with the parameters used and the reasons for each choice, and outline how the remaining stages will be evaluated.

5.1 Pipeline Overview

1. **Frame extraction (Phase 1)** — decode each video into JPEG frames at 30 frames per second.
2. **Garment segmentation and pose extraction (Phase 2, current)** — SAM2 segments the upper and lower garments in every frame from bounding-box prompts; MediaPipe extracts the body's pose landmarks in every frame.
3. **Point tracking (Phase 3)** — track points seeded on the garment masks across frames, tolerant to brief occlusion.
4. **Physics-driven cloth simulation (Phase 4)** — build a cloth mesh over the tracked points and simulate its expected motion, anchored to the real body motion from the pose landmarks, using a physics model of the cloth under gravity, stretch and damping forces.
5. **Feature extraction (Phase 4)** — compare the simulated cloth motion with the observed (tracked) cloth motion and compute physics-based features from that comparison.
6. **Classification (Phase 5)** — feed the features into a trained machine-learning classifier that outputs a real/AI-generated verdict and a confidence score.
7. **Reporting (Phase 5)** — show the verdict, confidence and the timestamps of the most physically implausible segments through a simple interface.

Figure 5.1 shows the same pipeline as a sequence of stages.

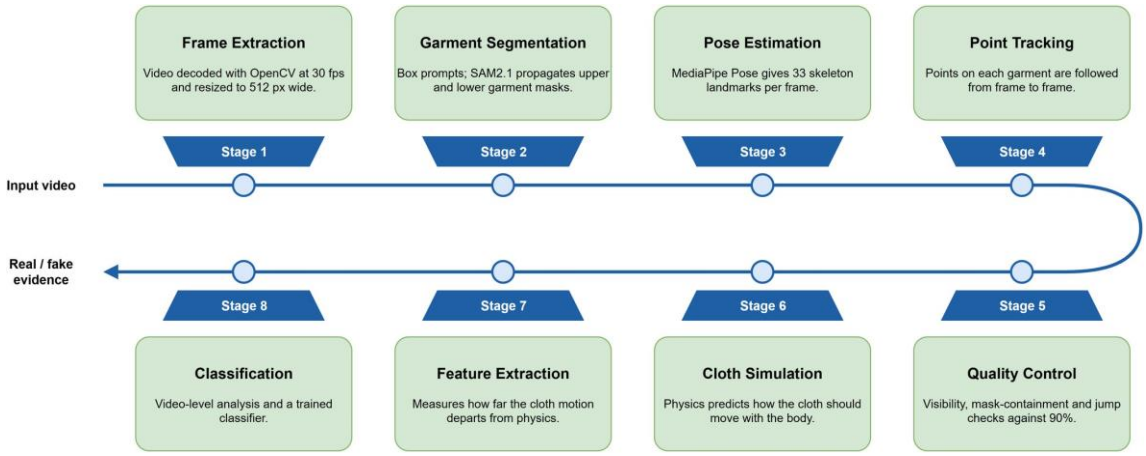


Figure 5.1: Overview of the Proposed Methodology

The eight stages of the pipeline, from the input video to the real/fake analysis. Stages 1–3 belong to Phases 1 and 2; the remaining stages are planned for Phases 3 to 5. Quality control of the tracked points is shown as its own stage.

5.2 Research Hypothesis

Most fake-video detectors learn the visual artefacts of particular generators: textures, blending edges or frequency patterns. Such detectors tend to fail when a new generator appears, because its artefacts differ. This project takes a different route. The motion of fabric is governed by physics: gravity pulls it down, it resists stretching, and it swings and settles after the body moves. Current video generators learn what cloth *looks like* from pixels, but they do not simulate the forces acting on it.

> **Hypothesis.** Given the real motion of a person's body, the motion of their clothing in a genuine video is predictable by physics, while in an AI-generated video it departs from that prediction. Because the test is based on physical law rather than on a particular generator's artefacts, it should apply to generators the system has never seen.

5.3 Dataset

Dataset methodology. The dataset is deliberately built to avoid shortcut learning: for every combination of clothing tightness (loose/tight) and motion type (e.g., walking, dancing, sports) included, both real and AI-generated examples are collected, across multiple different generators. This is intended to force the classifier to key on physical plausibility rather than on a particular content style, camera framing, or generator's cosmetic signature.

The current development dataset contains 14 videos of a single person in motion: 9 real videos and 5 AI-generated videos. It is used to build and check the pipeline. The evaluation dataset that the project will collect follows these criteria:

- loose, free-hanging clothing (dresses, skirts, loose shirts, scarves) as well as tight clothing, so that there is fabric for physics to act on;
- the subject filling the frame, with a torso at least about 150 pixels tall, so that measurement noise is small compared with the motion;
- matched framing and resolution between real and fake videos;
- at least 30 videos per class, with fakes from several different generators;
- similar kinds of motion in both classes.

5.4 Methodology per Stage

5.4.1 Frame Extraction

Frame extraction (Phase 1). Each video is decoded into individual JPEG frames at 30 frames per second, stored in one folder per video and named with plain zero-padded integers (for example 0000.jpg), the format SAM2's video loader requires.

Frames are resized to a standard width, keeping the aspect ratio, so that every video is processed at a similar size. The rate was first set to 10 frames per second for speed, but cloth swings and accelerates faster than a 100 ms gap between frames can capture, and later measurements of cloth motion need closely spaced samples, so Phases 1 and 2 were re-run at 30 frames per second.

5.4.2 Garment Segmentation

Garment segmentation (Phase 2). SAM2 is prompted with one bounding box around the upper garment and one around the lower garment, and propagates both masks through the video, giving a binary mask for each garment in every frame. Bounding boxes replaced single-point prompts, which give SAM2 no boundary information and sometimes produced whole-body or empty masks.

The system uses **SAM2.1** (Segment Anything Model 2) [7] in its video mode, in its small version so that it runs on a laptop GPU: given a prompt on one frame, it segments the object and propagates the mask through the rest of the video, keeping a memory of the object's appearance. The box prompts are drawn once per video on the first frame.

Figure 5.2 shows how the two box prompts lead to a garment mask, on which points are later followed from frame to frame.

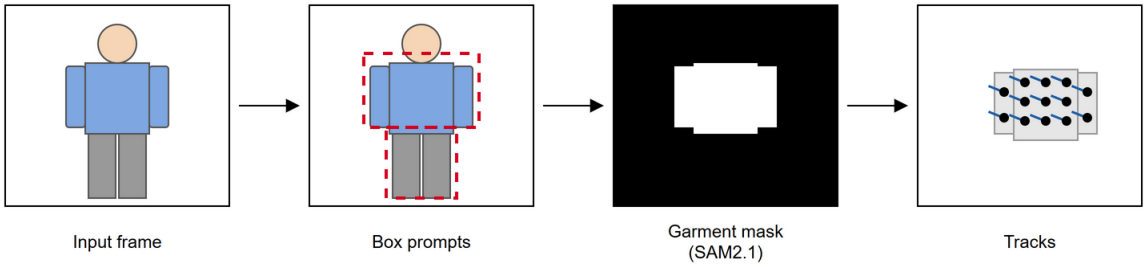


Figure 5.2: Garment Segmentation and Point Tracking

A box prompt gives a garment mask, and points on the garment are then followed from frame to frame.

5.4.3 Body Pose Estimation

Pose extraction (Phase 2). MediaPipe Pose is run on every frame to obtain the 33 body landmarks in pixel coordinates. These describe how the body under the clothing moves and later drive the cloth simulation.

MediaPipe Pose [8] returns 33 landmarks (shoulders, elbows, wrists, hips, knees, ankles and points on the face) for every frame. The landmarks are expressed in the same image coordinates as the garment masks.

5.4.4 Classification and Evaluation

Classification. The extracted per-segment and per-video features are aggregated and passed to a supervised classifier, trained and validated using video-level (not just segment-level) splits to avoid inflated, pseudo-replicated accuracy estimates. Model selection and evaluation will report standard discrimination metrics (e.g., AUC) alongside accuracy, and will specifically test whether performance holds on generators held out of training.

Segments from the same video are not independent, so the **video** will be the unit of evaluation: segment features are averaged per video, and every result is reported per video as well as per segment. Each feature's ability to separate the classes will be measured by the area under the ROC curve (AUC), with a Mann–Whitney U test for significance. Every feature will also be checked against **confounds**, the properties that differ between classes for reasons unrelated to physics, such as the subject's size in frame.

The proposed classifier is an L2-regularised logistic regression over the video-level features, evaluated by **leave-one-video-out** cross-validation: each video is predicted by a model trained on all the others. A simple, interpretable model is chosen deliberately, because the number of videos is small and because each coefficient can be explained in physical terms.

5.4.5 Explainability

Explainability. Alongside the verdict, the system retains the per-segment residual and feature values so that the timestamps and features driving a given decision can be reported back to the user, rather than only a final label.

5.5 Development Methodology

The project follows an **iterative, experiment-driven** process. Each phase moves through the same cycle: state a design decision and the reason for it before writing code; test it on one or two contrasting videos with both a visual overlay and a numeric check; run it on the full dataset; then audit the result before building on it. In Phase 2, for example, every video's masks and pose landmarks were checked visually before being accepted.

Every technical problem, with its cause and fix, is recorded in a troubleshooting log. Code is kept in a Git repository with one branch per phase, merged into the main branch when the phase is complete.

5.6 Conclusion

This chapter presented an approach to fake-video detection that tests whether clothing obeys physics. Each video is turned into frames at 30 frames per second, the garments are segmented with SAM2 from two box prompts, and the body's motion is captured with MediaPipe Pose. In the following phases, points on the garments will be followed through the video and their motion compared with what physics predicts for the same body movement. The resulting measurements, each with a physical meaning, will be passed to a classifier evaluated at the level of whole videos, and the time segments that drive each verdict will be reported back to the user. Chapter 6 describes how the system is designed.

Chapter 6 High-Level and Low-Level Design

This chapter describes the design of the system, first at a high level (its overall structure, layers and subsystems) and then at a low level (the sequence of interactions in each main operation). It also records the design considerations, architectural strategies, and the policies and tactics that shaped the implementation.

6.1 System Overview

The system is a **batch processing pipeline** that takes a folder of videos and will produce, for every video, a set of physics features and a real/AI-generated verdict. It is organised as a sequence of independent stages. The perception stages (Phases 1 and 2) are implemented; the motion, physics and decision stages are planned for Phases 3 to 5. Each stage is a Python program that reads the files written by the previous stage and writes its own files for the next one.

6.1.1 System Design

The pipeline has four groups of stages, each building on the last:

1. **Perception**: turns a video into frames, garment masks and body skeletons, using pretrained models (SAM2 and MediaPipe Pose).
2. **Motion**: places points on the garments, tracks them through the video and checks the quality of the tracks.
3. **Physics and features**: simulates how the cloth should move given the skeleton and computes the physics features.
4. **Decision**: classifies each video as real or AI-generated from its features, with a confidence score.

The reporting interface (Section 4.8) will run the same stages on a single uploaded video and present the result.

6.1.2 Data Flow

Every stage stores its output in its own area of the data store, organised by class and by video. Phases 1 and 2 store the frames, the garment boxes, the garment masks and the pose landmarks. The final outputs will be a feature table and the classifier's verdict for each video.

6.2 Design Considerations

This section describes the issues that had to be addressed before designing the system.

6.2.1 Assumptions and Dependencies

Assumptions

- Each video shows one person whose garments are visible for most of its length.
- The camera is still or moves slowly compared with the subject.
- The garment boxes drawn on the first frame are accurate enough for SAM2 to follow the garment.

Dependencies

- The pretrained models SAM2.1 and MediaPipe Pose, and their published checkpoints.
- A deep-learning framework build that matches the GPU driver installed on the machine.
- Compatible versions of the pose-estimation library and its supporting libraries.
- Google Colab, when no local GPU is available.

6.2.2 General Constraints

- **Hardware**: the target machine is a laptop-class GPU with around 8 GB of video memory, which is why the Small SAM2.1 model is used.
- **Platform**: development is on Windows with PowerShell, where file names with trailing spaces and locked folders caused failures.
- **Data**: the development dataset is small (14 videos), which rules out training large models and makes per-video evaluation necessary.
- **Privacy**: videos must not be uploaded to third-party services.

6.2.3 Goals and Guidelines

- **Generator independence**: the decision must rest on physical law, not on the artefacts of a particular generator.
- **Explainability**: every feature must have a physical meaning that a person can check on the video.

- **Fair comparison**: real and fake videos must receive exactly the same computation, and no measurement may depend on framing.
- **Reproducibility**: every result must be regenerable from the videos, the code and the logged parameters.

6.2.4 Development Methods

The project uses an iterative, experiment-driven method (Section 5.5). Each stage is checked on two contrasting videos with a visual overlay and a numeric check, then run on the whole dataset, then reviewed before the next stage is built on it. Work is version-controlled in Git, with one branch per phase.

6.3 System Architecture

Figure 6.1 shows the high-level architecture. The system is divided into six layers: a presentation layer through which the user operates it, four processing layers, and a data persistence layer that holds every intermediate result. The pretrained models are external components called by the perception layer.

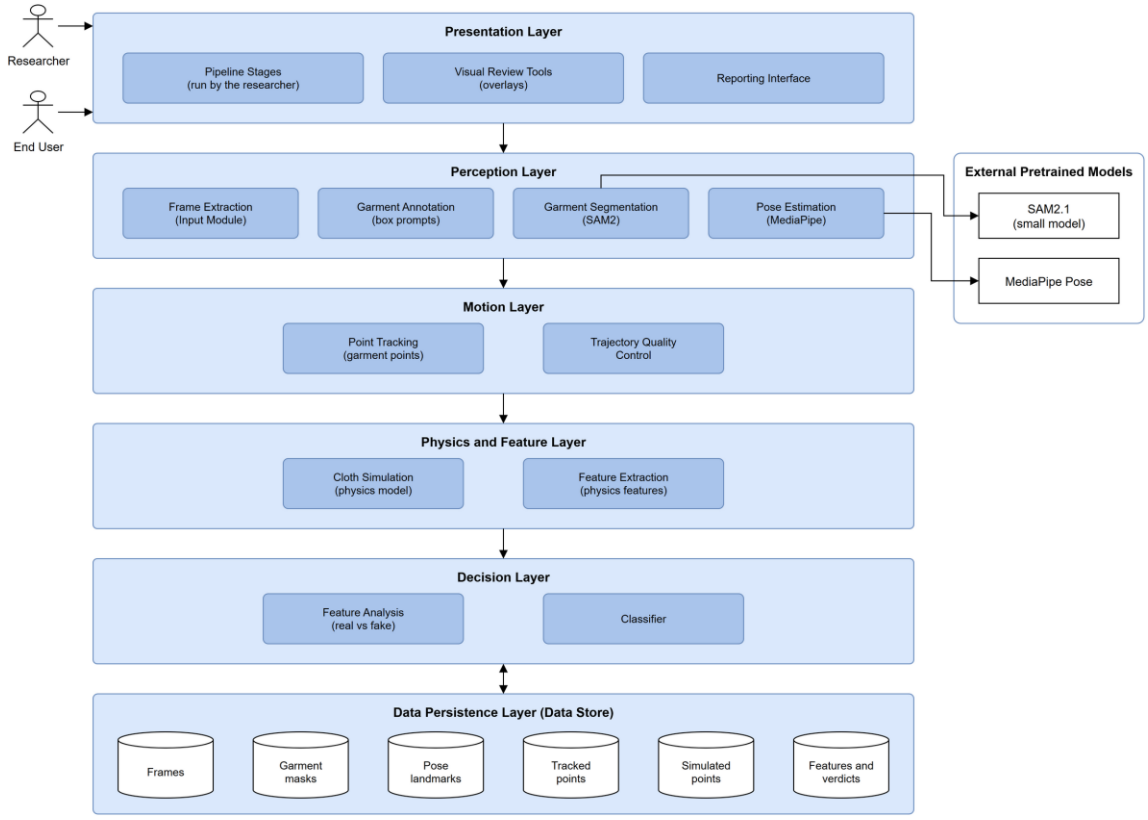


Figure 6.1: High-Level System Architecture

The presentation, perception, motion, physics and feature, and decision layers, the file-based persistence layer, and the external pretrained models.

6.3.1 Subsystem Architecture

Perception Subsystem

The perception subsystem (Figure 6.2) converts each video into numbered frames, obtains garment boxes from the user, segments the garments with SAM2 and estimates the skeleton with MediaPipe.

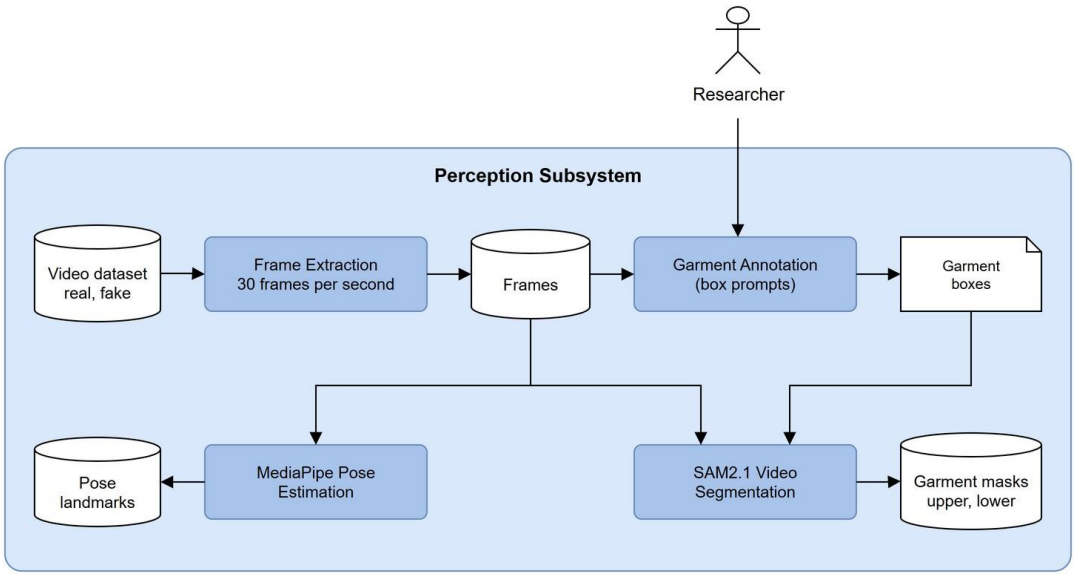


Figure 6.2: Perception Subsystem Architecture

Frame extraction, garment annotation, SAM2 segmentation and MediaPipe pose estimation, and the files each produces.

6.4 Architectural Strategies

The following strategies shaped the architecture of the system.

6.4.1 Pipe-and-Filter with File-Based Contracts

Each stage is a filter that reads stored data and writes its own, and the agreed structure of that data is the contract between stages. This has three benefits that mattered in practice: any stage can be re-run without repeating the expensive GPU stages before it; intermediate results can be inspected and visualised directly; and a stage can be replaced by another that writes the same format.

6.4.2 Technology Stack

- **Language**: Python, which offers the deep-learning, vision and scientific libraries the project needs.
- **Deep learning**: PyTorch, which SAM2 is built on.

- **Vision**: OpenCV for reading video and images and drawing overlays; MediaPipe for pose.
- **Numerical**: NumPy and SciPy for numerical work.
- **Visualisation**: Matplotlib for plots.

6.4.3 Reuse of Pretrained Foundation Models

Garment segmentation and pose estimation are solved with pretrained models rather than models trained for this project. The dataset is far too small to train such models, and pretrained models generalise across videos. The project's own contribution, the physics simulation and features, is a transparent model with a handful of physical parameters.

6.4.4 Offloading to the Cloud

The GPU stages can run on Google Colab through a single notebook that mounts Google Drive, so that data and checkpoints survive a disconnected session. Because the stages communicate through files, data produced on Colab is used by the local stages unchanged.

6.4.5 Error Handling and Recovery

Each stage processes videos one at a time and guards against errors. A failure is logged with the video's name and the program moves on. GPU memory is released after each video. Missing data (no pose in a frame, an empty mask) is recorded as missing rather than treated as an error.

6.4.6 Future Extension

The architecture allows the planned extensions without structural change: the Phase 5 classifier is a new stage that reads the feature table; the Phase 5 reporting interface is a new presentation component; and a 3D body or cloth model, or additional physical laws, would replace or add stages in the physics layer.

6.5 Component Diagram

Figure 6.3 shows the software components of Phases 1 and 2, the data they pass to each other, and the external libraries they use. The Input Module turns each video into frames; the Annotation Tool, Segmentation Module and Pose Extraction Module work from those frames; and the Visual Review Tools combine the garment masks and the pose landmarks into overlays for checking. All data passes through the data store.

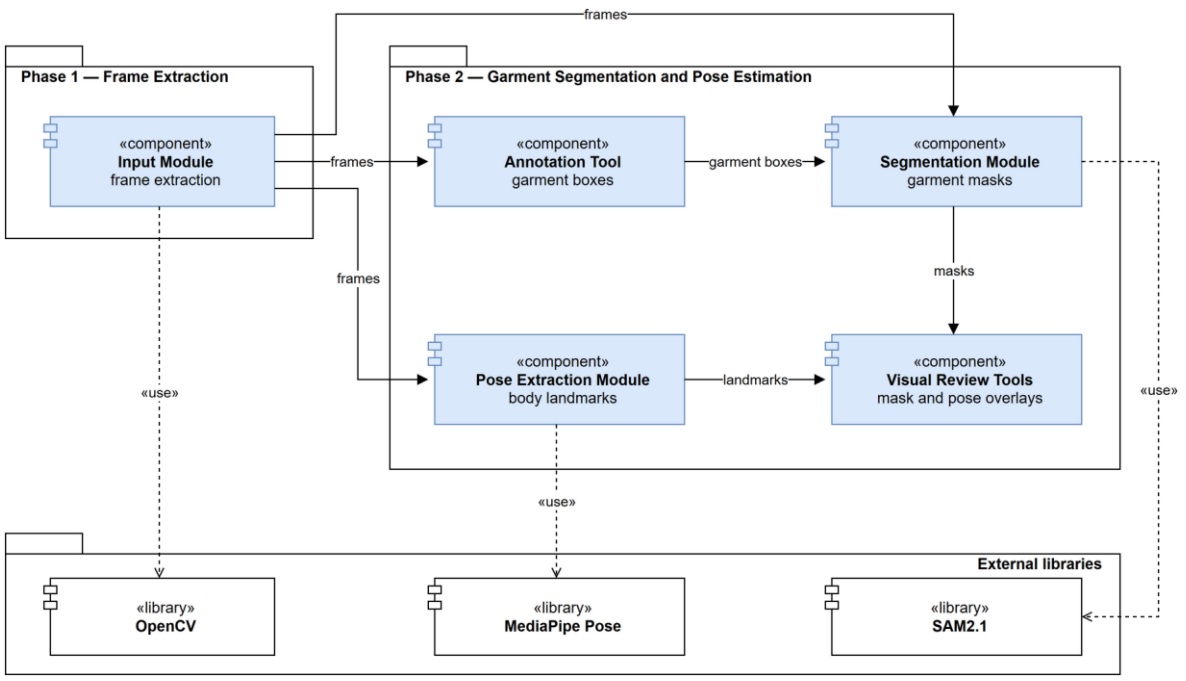


Figure 6.3: Component Diagram of Phases 1 and 2

The components of frame extraction (Phase 1) and of garment segmentation and pose estimation (Phase 2), the data each passes on, and the external libraries they use.

6.6 Sequence Diagrams

The following sequence diagrams show the interactions between the user, the system's modules, the pre-trained models and the data store in the main operations of the system. The end-to-end sequence covers the whole pipeline; the frame extraction sequence belongs to Phase 1, and the garment segmentation and pose estimation sequences belong to Phase 2.

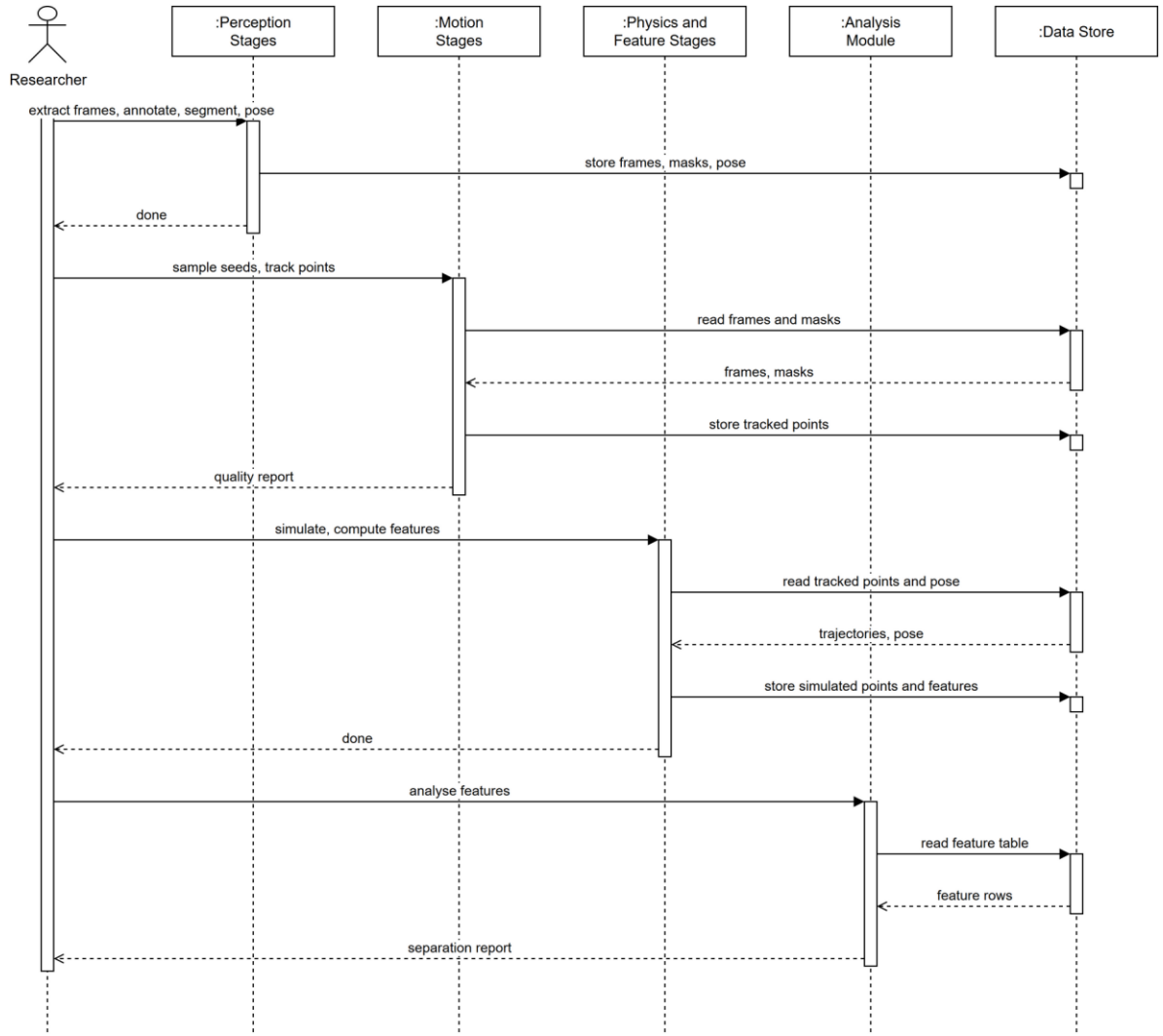


Figure 6.4: Sequence Diagram: End-to-End Pipeline (Planned Stages)

The researcher runs each stage in order; every stage reads the previous stage's files and writes its own.

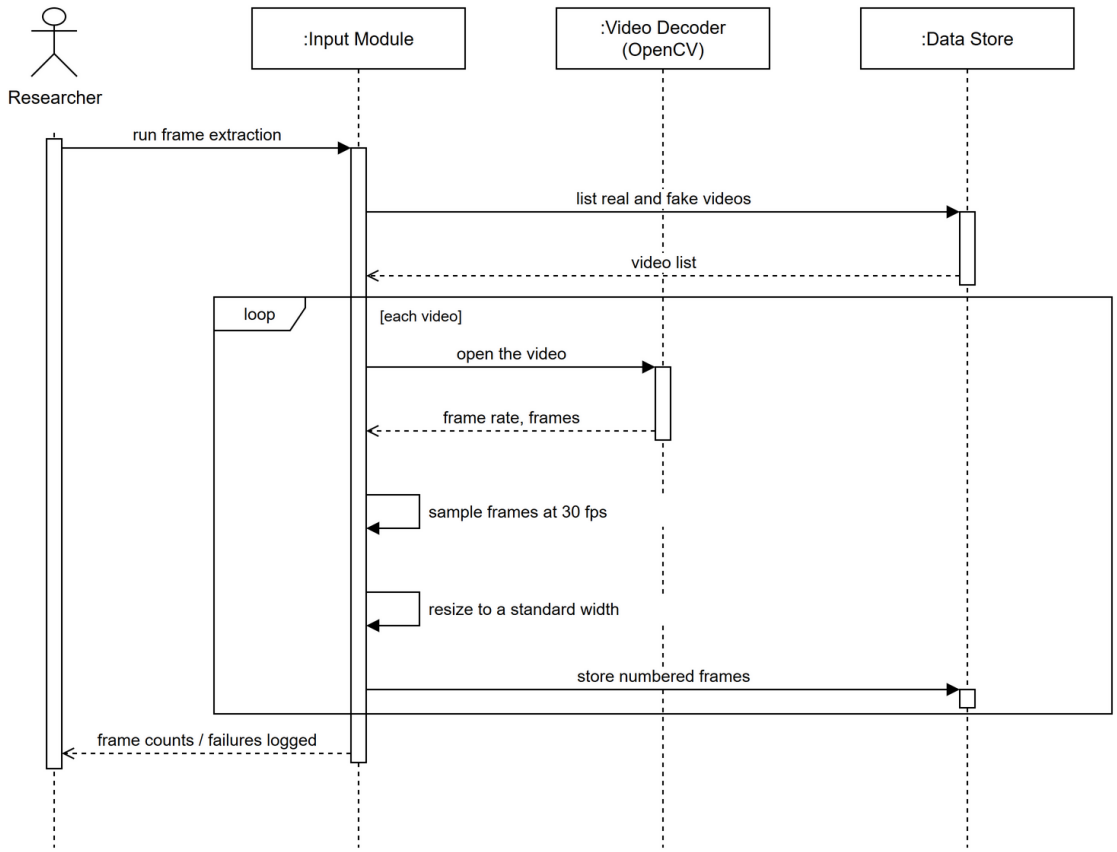


Figure 6.5: Sequence Diagram: Frame Extraction

For each video, the Input Module opens the file with the video decoder, samples its frames at 30 frames per second, resizes them to a standard width and stores them as a numbered sequence.

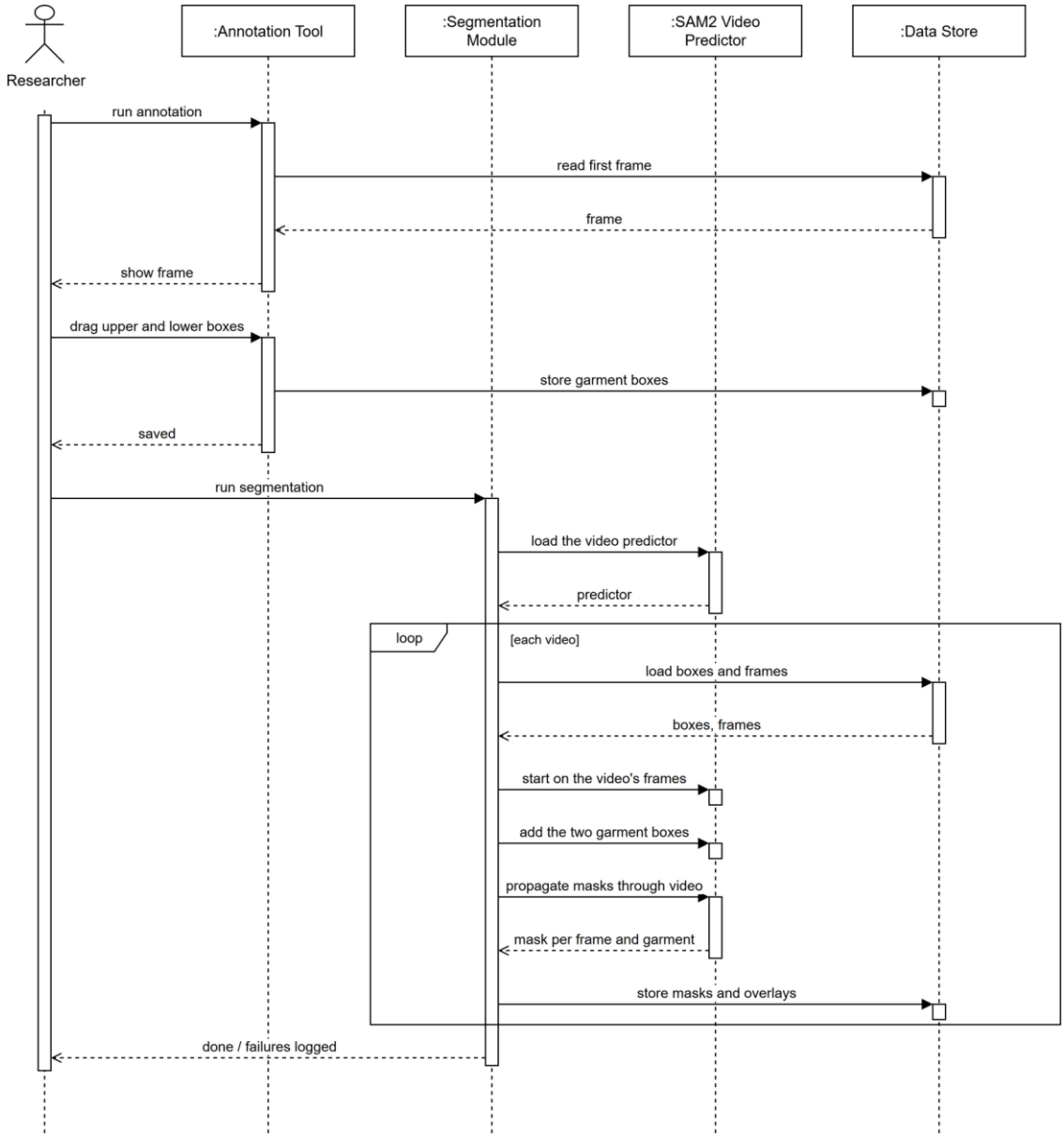


Figure 6.6: Sequence Diagram: Garment Segmentation

The researcher draws garment boxes; SAM2 is prompted with them on the first frame and propagates the masks through the video.

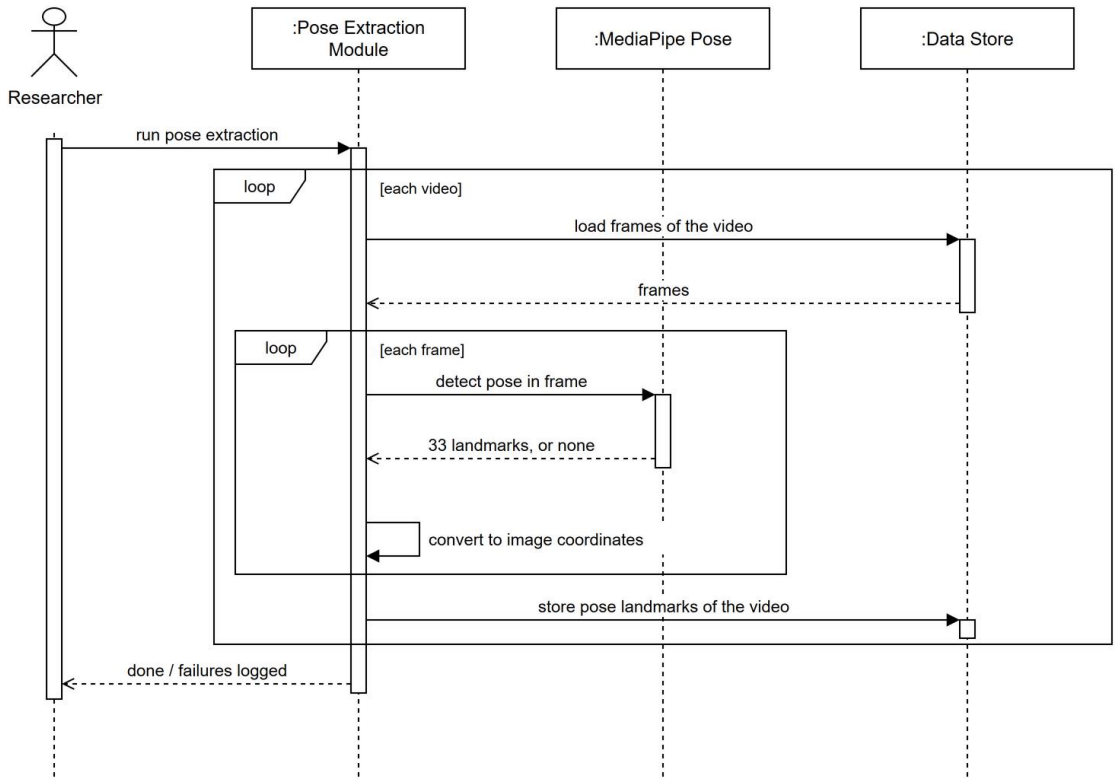


Figure 6.7: Sequence Diagram: Pose Estimation

For each video, every frame is passed to MediaPipe Pose, which returns 33 body landmarks (or none when no person is found); the landmarks are converted to image coordinates and stored.

6.7 Policies and Tactics

This section describes the design policies and tactics used in the implementation of Phases 1 and 2. Unlike the architectural strategies, which shape the system as a whole, these decisions affect how individual modules are built. The main ones are:

1. Frame rate of 30 frames per second
2. Box prompts for segmentation
3. Visual review of masks and pose
4. Coding guidelines

6.7.1 Frame Rate of 30 Frames per Second

Frames are extracted at 30 frames per second. A first version used 10 frames per second for speed, but cloth swings and accelerates faster than a 100 ms gap between frames can capture, so the rate was raised and Phases 1 and 2 were re-run.

6.7.2 Box Prompts for Segmentation

SAM2 is prompted with a bounding box per garment rather than a single point. A point gives no boundary, and point prompts produced masks of the whole body or empty masks; boxes produced reliable masks.

6.7.3 Visual Review of Masks and Pose

Every later stage inherits the output of Phase 2, so a garment mask that misses the cloth or a misplaced body joint would become an error in everything built on it. The masks and pose landmarks of every video are therefore drawn over the frames and reviewed before they are accepted.

6.7.4 Coding Guidelines

The code follows the standard Python style guide (PEP 8), and every module begins with a description of what it does, why it is designed that way and how to run it. Every parameter that affects the results is configurable and has a documented default.

6.8 Conclusion

This chapter presented the design of the system. At a high level, it is a pipeline of independent stages in six layers that communicate through stored data, which makes every stage replaceable, re-runnable and inspectable. Phase 1 is complete and Phase 2, which completes the perception layer, is at the point of completion: their components are shown in the component diagram and their interactions in the frame extraction, garment segmentation and pose estimation sequence diagrams. The policies and tactics recorded here, from the 30 frames-per-second rate and box prompts to the visual review of every video's masks and pose, ensure that the frames, garment masks and body landmarks handed to the later phases are reliable. The design leaves room for the remaining phases without structural change.

Bibliography

[1] S. Motamed, L. Culp, K. Swersky, P. Jaini, and R. Geirhos, “Do generative video models understand physical principles?,” *arXiv preprint arXiv:2501.09038*, 2025.

[2] M. Sugiyama and H. Kataoka, “Simple visual artifact detection in Sora-generated videos,” *arXiv preprint arXiv:2504.21334*, 2025.

[3] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, “FaceForensics++: Learning to detect manipulated facial images,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[4] A. Sarkar, H. Mai, A. Mahapatra, S. Lazebnik, D. A. Forsyth, and A. Bhattad, “Shadows don’t lie and lines can’t bend! generative models don’t know projective geometry. . . for now,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[5] D. Baraff and A. Witkin, “Large steps in cloth simulation,” in *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pp. 43–54, 1998.

[6] M. Müller, B. Heidelberger, M. Hennix, and J. Ratcliff, “Position based dynamics,” *Journal of Visual Communication and Image Representation*, vol. 18, no. 2, pp. 109–118, 2007.

[7] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, “SAM 2: Segment anything in images and videos,” *arXiv preprint arXiv:2408.00714*, 2024.

[8] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, “BlazePose: On-device real-time body pose tracking,” *arXiv preprint arXiv:2006.10204*, 2020.